3 Treaps: Using Randomization to Balance Binary Search Trees

This chapter covers

- Problem: index data according to multiple criteria
- The Treap data structure
- Problem: keep a binary search tree balanced
- Using treaps to implement balanced binary search trees (*BST*)
- Randomized Treaps (*RT*)
- Comparing plain BSTs and RTs

In chapter 2 we have seen how it is possible to store elements and retrieve them based on their priorities by using *heaps*, and how we can improve over binary heaps by using a larger branching factor.

Priority queues are especially useful when we need to consume elements in a certain order from a dynamically changing list (like the list of tasks to run on a CPU), so that at any time we get the next element (according to a certain criterion), remove it from the list and (usually) not worry anymore about fixing anything for the other elements. The difference with a sorted list is that we only